Implemetando Dicionários com Árvore AVL, Árvore Vermelho-Preto, Tabela Hash com Encadeiamento Interior e Exterior

Jose N. Bernardo N. Filho[1]

[1]Campus Quixadá – Universidade Federal do Ceará (UFC)

Abstract. *This paper presents dictionary implementations using four data structures: AVL tree, red-black tree, hash table with internal chaining, and hash table with external chaining (double hash). For each approach, we discuss in detail the balancing algorithm, the basic operations (insertion, removal, and search), and mechanisms for handling collisions or imbalances, as well as performance metrics such as comparison count and rotation count.*

Resumo. *Este trabalho academico apresenta implementações de dicionários utilizando quatro estruturas de dados: árvore AVL, árvore vermelho-preto, tabela hash com encadeamento interno e tabela hash com encadeamento externo (hash duplo). Para cada abordagem, discutimos detalhadamente o algoritmo de balanceamento, as operações básicas (inserção, remoção e busca) e mecanismos de tratamento de colisões ou desequilíbrios, além de métricas de desempenho, como contagem de comparações e rotações.*

1. Introdução aos Dicionários

Dicionários são estruturas de dados que armazenam pares de chave-valor, permitindo associar uma chave a um valor correspondente. Eles são amplamente usados por sua eficiência e flexibilidade, especialmente quando se precisa acessar ou modificar dados com base em uma identificação específica (a chave), em vez de um índice como em listas ou `arrays`. Minhas implementações foram baseadas nos Dicionários do `c++`, como o `Map` e o `Unordered Map`. Não optei pelo tratamento de erros durante o código porque essas estruturas de dados no `c++` não trata suas funções dessa maneira. Todas as implementações que eu fiz foram feitas para todo tipo de pares de valores por causa da implementação de dados genéricos.

2. Dicionários com Árvore AVL

Uma árvore AVL é um tipo de árvore binária de busca que possui um mecanismo de balanceamento baseado nas alturas das subárvores de um nó. Uma regra da árvore AVL é manter a diferença de altura entre as subárvores esquerda e direita de qualquer nó limitada a no máximo um. Esse equilíbrio é essencial para garantir que as operações básicas da estrutura, como inserção, remoção e busca, ocorram com eficiência em tempo logarítmico. Quando essa diferença ultrapassa o limite permitido após alguma modificação na árvore, são feitas rotações que reorganizam a árvore, restaurando o equilíbrio. Isso torna as árvores AVL ideais para situações em que as buscas são mais frequentes que as inserções e remoções, pois elas garantem tempos de acesso consistentes e rápidos, mesmo que o custo para inserções e remoções seja maior devido ao balanceamento restrito, que ao remover ou inserir um nó, pode haver várias rotações para manter a estrutura da árvore.

2.1. Mantendo a árvore AVL balanceada

De maneira simples, quando uma árvore AVL está desbalanceada, quer dizer que a diferença da altura entre a subárvore da esquerda e direita é dois. Desse modo, é feita uma rotação para que a diferença das alturas seja de no máximo um ou zero. Para isso acontecer, é preciso analisar vários casos, são eles:

Quando a subárvore da esquerda é mais alta que a subárvore da direita, dizemos que houve um desbalanceamento à esquerda. Para corrigir isso, é possível subir um nó da subárvore esquerda e descer o nó que estamos analisando, de forma que a árvore volte a ficar balanceada. Esse processo é chamado de rotação à direita.

Já quando a subárvore da direita é mais alta que a subárvore da esquerda, Temos um desbalanceamento à direita. Nesse caso, fazemos o oposto: um nó da subárvore direita sobe, e o nó que causou o desbalanceamento desce. Essa operação é chamada de rotação à esquerda.

Existem também situações em que uma simples rotação não resolve o problema. Nesses casos, é necessário fazer duas rotações: uma para preparar a árvore e outra para balanceá-la. Essas situações ocorrem quando a inserção ou remoção acontece "no lado oposto" da subárvore mais alta, e os casos são resolvidos com rotações duplas (esquerda-direita ou direita-esquerda).

2.2. Implementação do Dicionário com uma AVL

Normalmente, nas implementações das AVL's não existe um ponteiro para pai, sendo mais comum nas árvores vermelho-preto. Optei pelo uso de ponteiros para pai por causa da facilidade de usar o iterador ou `iterator` na estrutura. Nas estruturas de dados, um Dicionário é chamado de Map. Foi incluída a biblioteca `std::utility` onde está presente o `std::pair` para guardar chaves-valores na árvore.

- **Classe Node**: Começando pela estrutura de um nó da AVL, além de seus atributos básicos, ele tem um ponteiro para pai e um conjunto chave-valor para ser implementado no Map. Na classe está presente também uma declaração `friend class` para as classes do iterador e da estrutura principal(Map). Dessa forma, ambas as classes têm acessos aos atributos e os métodos da classe Node.
- **Classe Map**: Além dos métodos comuns para a AVL(calcular altura, rotacionar um nó, buscar um valor, etc...) vistos em aulas, implementei uma remoção recursiva e uma sobrecarga de operador [].
  - **Remoção recursiva**: Optei pela remoção recursiva por sua fácil implementação quando o nó a ser removido tem dois filhos. Quando isso acontece, seu valor é substituído com o seu sucessor. No meu caso, usei o menor valor da subárvore da direita. Desse modo, somente é necessário chamar recursivamente para o nó que agora está com um valor repetido na arvore e remove-lo usando como argumento o seu valor, caindo em um dos casos anteriores.
  - **Sobrecarga de operador []**:Em C++ é possível sobrecarregar operadores em diversas classes. No caso do `operator[]`, ele serve tanto para acessar quanto para substituir valores. Na implementação, recebe-se apenas

a chave como argumento; se essa chave já existir na AVL, retorna-se referência ao valor correspondente; caso contrário, insere-se um novo elemento com o valor padrão e retorna-se referência a esse novo valor. Como o operador retorna referência, pode-se utilizá-lo em conjunto com o operador de atribuição (=) para sobrescrever o valor existente.

- **Classe Iterator**: Para ter um iterador na árvore, é preciso ter um método que pegue o sucessor de um nó para combinar com o `operator++`. Para isso ocorrer, precisamos de um método que calcula o sucessor de qualquer nó.
  - **Sucessor**: Se dado nó tiver uma subárvore á direita, seu sucessor será o menor nó da subárvore da direita. Se um nó não tiver nenhuma subárvore a esquerda, o melhor a se fazer é subir na arvore até encontrar um ancestral em que seu último filho não seja seu filho direito.

  Dessa forma, o `operator++` percorre o próximo nó na árvore em ordem. Outro operador implementado é o desreferenciador, `operator*`, que retorna uma referência ao par chave–valor, permitindo o acesso direto ao valor associado. De modo semelhante, o `operator->` retorna um ponteiro para o objeto `std::pair`, possibilitando o acesso direto aos seus membros.

2.3. Contadores

Foram implementados dois tipos principais de contadores:

- **Contador de comparações**: incrementado sempre que duas chaves são comparadas durante operações de busca, inserção ou remoção. Serve para medir a eficiência das operações de pesquisa na árvore.
- **Contador de rotações**: incrementado a cada rotação simples ou dupla executada para manter o balanceamento estrito da AVL. Permite avaliar o impacto do balanceamento na performance da estrutura.

3. Dicionários com Árvore Vermelho-Preta

Uma árvore vermelho-preta (RBT - Red-Black Tree) é um tipo de árvore binária de busca balanceada que mantém seu equilíbrio por meio de propriedades associadas às cores dos nós. Cada nó pode ser vermelho ou preto, e a árvore segue cinco propriedades que garantem sua altura logarítmica, mesmo após operações de inserção e remoção. Essas propriedades evitam que o caminho mais longo da raiz até uma folha seja mais que o dobro do caminho mais curto, o que garante operações eficientes de busca, inserção e remoção.

O balanceamento em uma árvore vermelho-preta é mais flexível do que em uma AVL, o que torna as inserções e remoções menos custosas, em média, em termos de rotações. O custo disso é um tempo de busca um pouco menos previsível, mas ainda eficiente. As RBTs são bastante utilizadas em bibliotecas padrão, como em `std::map` e `std::set` da STL do C++.

3.1. Balanceamento na RBT

Diferente da AVL, o balanceamento em uma árvore vermelho-preta é feito por meio da alteração das cores dos nós e, quando necessário, por rotações simples ou duplas. As regras principais que regem esse balanceamento são:

- Todo nó é vermelho ou preto;

- A raiz é sempre preta;
- Todas as folhas (nulos) são pretas;
- Se um nó é vermelho, então seus filhos são pretos (propriedade de não haver dois vermelhos consecutivos);
- Para cada nó, todos os caminhos da raiz até as folhas descendentes contêm o mesmo número de nós pretos.

Após inserções ou remoções, essas propriedades podem ser violadas. Para restaurá-las, utiliza-se o método `fixNode`, que executa recolorizações e rotações com base nos casos descritos no livro de Cormen.

3.2. Implementação do Dicionário com uma RBT

A estrutura da árvore vermelho-preta é bastante similar à da AVL, com a principal diferença sendo no método de balanceamento. Portanto, muitas das estratégias e padrões usados na implementação da AVL foram reaproveitados.

- **Classe Node**: Assim como na AVL, a classe de nó da RBT possui ponteiros para pai, filho esquerdo e direito, além do campo de chave-valor (usando `std::pair`). Adicionalmente, possui um atributo para a cor do nó. Também é usada a declaração `friend class` para que as classes `Map` e `Iterator` possam acessar seus membros privados, facilitando a navegação e manipulação da estrutura.
- **Classe Map**: Seguindo a mesma ideia da AVL, a classe principal implementa os métodos tradicionais de uma árvore binária de busca, com destaque para:
  - **Inserção com Balanceamento**: Após inserir um nó, pode haver violação das propriedades da RBT. O método `fixNode` é utilizado para corrigir a estrutura, realizando recolorações e rotações conforme necessário.
  - **Remoção com Balanceamento**: A remoção segue a mesma lógica da AVL, substituindo o nó com dois filhos por seu sucessor. A diferença está na fase de balanceamento, que utiliza o método `fixNode` para restaurar as propriedades da árvore. A implementação segue os casos descritos no livro de Cormen.
  - **Sobrecarga do operador []**: Da mesma forma que na AVL, o operador `[]` foi sobrecarregado para acessar ou inserir pares chave-valor. Caso a chave já exista, retorna uma referência para o valor associado; se não, insere um novo par com valor padrão e retorna a referência para esse valor.
- **Classe Iterator**: O iterador foi implementado de forma idêntica à AVL, já que a estrutura de navegação da árvore é semelhante. O ponteiro para pai permite caminhar na árvore e implementar o operador ++ usando o conceito de sucessor.
  - **Sucessor**: A função de sucessor segue a mesma lógica da AVL: se um nó possui subárvore à direita, o sucessor é o menor valor dessa subárvore. Caso contrário, percorre-se o caminho para cima até encontrar um ancestral que tenha o nó atual como filho à esquerda.

  O operador $\star$ retorna uma referência ao par chave-valor do nó, enquanto o operador $->$ retorna um ponteiro para o par, permitindo acesso direto aos elementos como em estruturas tradicionais.

3.3. Contadores

Foram implementados dois tipos principais de contadores:

- **Contador de comparações**: incrementado a cada comparação de chaves durante operações de busca, inserção ou remoção, permitindo avaliar a eficiência das pesquisas na árvore.
- **Contador de rotações**: incrementado a cada rotação (simples ou dupla) executada para manter o balanceamento estrito da AVL. Essa métrica é fundamental para comparar o desempenho de balanceamento entre AVL e RBT, já que árvores rubro-negras exigem, em geral, menos rotações.

4. Dicionários com Tabela Hash com Encadeamento Interior

Uma tabela hash é uma estrutura de dados que permite o acesso eficiente a elementos através de uma função de dispersão (hash), que transforma uma chave em um índice dentro de um vetor. Quando ocorre uma colisão — ou seja, quando duas chaves diferentes mapeiam para o mesmo índice — uma estratégia comum é o uso de encadeamento interior, também conhecido como hashing com listas encadeadas. Nesse método, cada posição do vetor armazena uma lista de pares chave-valor.

As operações de busca, inserção e remoção tem desempenho esperado constante, desde que a função hash seja eficiente e a tabela esteja com um fator de carga controlado. Para manter essa eficiência, foi utilizado um `max load factor`, que define o limite da média de elementos por posição antes que a tabela seja redimensionada.

4.1. Implementação do Dicionário com Tabela Hash

A implementação utiliza recursos da biblioteca padrão do C++, com destaque para os seguintes elementos:

- **`std::vector`**: A tabela em si é implementada como um vetor de listas. Cada posição do vetor corresponde a um bucket da tabela hash.
- **`std::list`**: Cada bucket contém uma lista de pares chave-valor (`std::pair`), utilizada para tratar colisões via encadeamento.
- **`std::hash`**: A função de dispersão utilizada é a padrão do C++, definida na biblioteca `<functional>`.
- **Encapsulamento de pares chave-valor**: Os elementos da tabela são armazenados como `std::pair<Key, Value>` da biblioteca `<utility>`.
- **Ponteiros ao invés de iteradores**: Diferente das implementações com árvores (AVL e RBT), onde o iterador é importante para a navegação ordenada, nesta estrutura optei por utilizar ponteiros simples para os pares dentro das listas. Isso facilita a implementação das operações básicas, especialmente as funções `find` e `remove`, que retornam ponteiros para os elementos encontrados ou `nullptr` caso não existam.
- **Sobrecarga do operador `[]`**: A lógica segue o mesmo padrão das árvores. Caso a chave já esteja presente, é retornada uma referência para seu valor. Caso contrário, um novo par com valor padrão é inserido, e uma referência a esse novo valor é retornada. A inserção pode desencadear um *rehash* caso o fator de carga máximo seja ultrapassado.

4.2. Redimensionamento da Tabela (Rehash)

Quando o número de elementos inseridos excede o limite imposto pelo `max load factor`, a tabela é redimensionada. O processo de rehash consiste em:

1. Encontrar o próximo número primo maior que o dobro do tamanho atual da tabela. Esse valor é usado como novo tamanho da tabela para manter uma boa distribuição dos elementos.
2. Recalcular o índice de hash de cada par chave-valor, inserindo-os novamente na nova tabela.

Para garantir eficiência, a função que encontra o próximo primo foi implementada com complexidade $O(\sqrt{n})$, permitindo um redimensionamento relativamente rápido mesmo para tabelas grandes.

4.3. Contadores

Foram implementados dois tipos principais de contadores:

- **Contador de comparações**: Cada vez que uma chave é comparada com outra durante uma busca, inserção ou remoção, esse contador é incrementado. Ele serve como uma medida prática de eficiência da função hash e da distribuição dos dados.
- **Contador de rehashes**: Armazena a quantidade de vezes que a tabela foi redimensionada durante o uso. Isso permite avaliar como o *load factor* e a função hash estão influenciando o desempenho da estrutura.

5. Dicionários com Tabela Hash com Encadeamento Exterior (Hash Duplo)

No encadeamento exterior, também conhecido como endereçamento aberto, as colisões são tratadas realocando elementos diretamente dentro da própria tabela hash, sem o uso de listas encadeadas. Dentre os métodos de resolução de colisões, o **hash duplo** (*double hashing*) é um dos mais eficientes, pois reduz significativamente o agrupamento de elementos em regiões da tabela.

Diferente do encadeamento interior, onde cada posição da tabela pode armazenar múltiplos elementos via listas, no encadeamento exterior cada posição armazena no máximo um único elemento. Quando há colisão, uma nova posição é calculada com base em uma função de dispersão secundária, produzindo um novo deslocamento para cada tentativa.

O hash duplo proporciona n^2 combinações distintas para a sondagem, onde n é o tamanho da tabela, o que oferece uma cobertura muito maior do que a sondagem linear ou quadrática, que produzem apenas n deslocamentos distintos.

5.1. Implementação do Dicionário com Hash Duplo

A estrutura foi implementada com os seguintes elementos da linguagem C++:

- **Uso de `std::vector`**: A tabela é representada como um vetor de pares. Cada posição armazena um par composto por:
  - Um caractere (definido com `#define`) que indica a **situação da posição**: se está vazia ou ocupada. Essa abordagem permite o controle eficiente do estado de cada célula.

  - Um par `(chave, valor)` implementado com `std::pair` da biblioteca `<utility>`.
- **Uso de ponteiros**: Assim como na implementação com encadeamento interior, optou-se por utilizar ponteiros para facilitar a manipulação e retorno dos valores. As funções `find` e `remove` retornam ponteiros para os elementos encontrados, ou `nullptr` caso a chave não exista.
- **Sobrecarga do operador `[]`**: Foi implementada da mesma maneira que nas outras estruturas. Caso a chave exista, retorna-se uma referência para seu valor. Caso contrário, insere-se um novo par com valor padrão e retorna-se uma referência para esse valor. A operação pode disparar um *rehash* caso o fator de carga máximo seja excedido.
- **Hash duplo**: Para calcular a posição de um elemento, são utilizadas duas funções de hash:
  - **Hash primária**: baseada no tamanho atual da tabela (um número primo).
  - **Hash secundária**: baseada no maior primo menor que o tamanho da tabela.

  Ambas as funções foram implementadas com complexidade $O(\sqrt{n})$, garantindo bom desempenho mesmo para tabelas grandes.

5.2. Redimensionamento da Tabela (Rehash)

Quando o fator de carga ultrapassa o limite estabelecido, é necessário realizar o redimensionamento da tabela. O processo envolve:

1. Calcular o próximo número primo maior que o dobro do tamanho atual da tabela.
2. Atualizar a função hash secundária para o novo maior primo menor que o novo tamanho.
3. Recalcular e reposicionar todos os pares chave-valor na nova tabela.

5.3. Contadores

Foram utilizados os mesmos contadores presentes nas demais implementações:

- **Contador de comparações**: Incrementado sempre que uma chave é comparada durante uma operação de busca, inserção ou remoção.
- **Contador de rehashes**: Indica quantas vezes a tabela precisou ser redimensionada ao longo da execução.

6. Mudanças do envio anterior

No primeiro envio, tiveram algumas funções faltantes para completar as estruturas de dados: RBT e hash com endereçamento aberto. Sem essas funções, a RBT não ficava com suas propriedades quando acontecia alguma remoção e o hash com endereçamento aberto fazia uma busca muito lenta. Outro caso foi o tamanho das tabelas hashes que começavam com valores muito altos, fazendo com que quase não houvesse colisões durante a execução. Vou discutir as partes detalhadamente nos tópicos a seguir.

6.1. Função de consertar a RBT depois da remoção

A função `fixDeleteNode()` agora cobre os casos que a remoção pode causar na RBT, que é no caso o duplo preto. Da mesma forma do `fixNode()` para a inserção, eu utilizei como referência os pseudocódigos do livro do Cormen. Desse modo, agora o dicionário consegue lidar com as remoções que acontecem durante a execução.

6.2. Falta de um slot `DELETED` na OpenHash

Um erro gravíssimo foi no hash com endereçamento aberto não ter um slot com uma marcação que foi deletado. O principal problema era quando buscava um elemento no OpenHash. Na busca, o pior caso percorria a tabela inteira, e, como no operador `[]`(colchetes) acontecem duas buscas, acabava que a inserção dos dados demorava muito tempo na tabela. Um exemplo era ao ler o livro 'O alienista' do Machado de Assis, onde o tempo total de inserção de todas as palavras era de aproximadamente 55 segundos. Em breve iremos discutir mais sobre essa métrica e falaremos o porquê desse valor ser não um bom sinal. A única solução possível era colocar o slot `DELETED` na tabela hash, sendo necessário mudar algumas funções, dentre elas: `_find()`, `_insert()`, `_erase()`.

6.2.1. Função `find`

A função de encontrar um elemento retorna um ponteiro nulo assim que encontra um slot `EMPTY`. Esse modo é eficiente porque não é preciso percorrer a tabela inteira. Com o novo slot, se houve uma colisão e os elementos que forem inseridos anteriormente forem removidos, eles serão tratados da mesma forma do `BUSY`, ou seja, só é necessário verificar se o slot é vazio para retornar o nulo.

6.2.2. Função `insert`

Diferente da função anterior, agora existe uma bandeira para o primeiro slot deletado na tabela. Dessa forma, é garantido que não se inserirá na tabela duas chaves de mesmo valor. Ao encontrar o primeiro slot vazio, é inserido ou no primeiro slot deletado, se existir, ou no slot vazio. A função tem também um retorno da posição que foi inserido o elemento para auxiliar no operador `[]`(colchetes).

6.2.3. Função `delete`

A única diferença é algo que acontece na função de inserir, onde é possível retornar nulo se encontrar `EMPTY`; caso contrário, se o valor for encontrado, ele recebe a bandeira `DELETE`.

6.2.4. Operador `[]`

Na função anterior, era usado retorno de ponteiros para buscar e inserir valores. Como o uso de ponteiros pode ocasionar acesso inválido na memória, refiz a função de modo que ela faz a busca manual e usa a função de inserção. Como a função de inserção retorna a posição onde foi inserido, o operador de colchetes consegue retornar a variável do valor em tempo constante.

6.3. Cálculo de posição no operador `[]` no Hash Fechado

Na função anterior, se ocorresse um rehash na inserção do valor, a posição antiga seria inválida para a nova tabela hash, resultando em um acesso inconsistente. Com isso, foi necessário recalcular a posição depois da inserção na tabela.

7. Teste de eficiência com leitura de arquivos txt

Foram feitos testes com alguns arquivos `txt` onde era lido o texto completo e contava a frequência das palavras. Como os textos lidos foram livros completos, os testes de eficiência consistiam em contar o tempo para colocar a frequência de todas as palavras do livro. Os dicionários criados foram do tipo chave-string e valor-inteiro. Como anteriormente foi feito o operador de colchetes em todas as implementações, para contar a frequência de palavras era só utilizar:

```
map[chave]++;
```

onde se a chave estivesse no `map`, ela seria incrementada; caso contrário, era criada no `map` com o valor zero e depois incrementada. Outro detalhe é que a implementação foi construída de forma que fosse quase universal em quase todas as estruturas, a única restrição foi que as implementações da AVL e RBT não eram compatíveis com as de Hash com encadeamento interior e exterior e vice-versa. Portanto, as `main.cpp` de AVL e RBT são as mesmas, bem como as de Hash com encadeamento interior e exterior.

7.1. Leitura dos textos `txt`

Como alguns textos utilizados para fazer as análises tinham caracteres especiais, foi necessário usar a biblioteca ICU(International Components for Unicode). Ao final desse documento, são fornecidas todas as instruções de instalação necessárias para executá-lo na sua máquina. Na biblioteca, é fornecida uma bandeira que indica se um processo teve erro ou não. Esse é o passo inicial para utilizar qualquer função que a ela implementa. Dentro da ICU existe uma `RegexPattern`, que seria um padrão de escrita para as palavras do texto. Nesse padrão, é passado como argumento tudo que deve ser incluído como palavra, inclusive:

- Hífen no meio da palava
- Qualquer trecho que não tenha espaços
- Palavras com acentos(já que normalmente eles não são incluídos)

É importante ressaltar que ”palavras” que começam com números, ou que há hífen ou traços no final ou começo não são consideradas no código. Os argumentos passados no padrão de escrita já lidam com isso. O processo para inserir palavras nos dicionários se baseia no padrão de escrita. Os passos para coletar cada palavra do texto e converter para o padrão mencionado serão descritos a seguir:

- Cada linha do texto é armazenada em uma `string`.
- A linha é convertida para um tipo de `string` que a biblioteca ICU consegue manusear.
- Cada palavra da linha será convertida para o padrão de escrita e depois colocada em um vetor de `string` do tipo ICU(mais especificamente `unicodeString`)

Esse vetor será iterado e, em seguida, são inseridos todos os seus elementos no dicionário. Nessa parte, é contabilizado o tempo de inserção do texto inteiro.

Foi usada a biblioteca `fstream`, onde é possível abrir arquivos `txt`, tendo como parâmetro o diretório. É possível também criar arquivos `txt` em diretórios pré-estabelecidos.

7.2. Métrica de eficiência

Para cada implementação de dicionário, foi utilizado o tempo de cada um dos processos a seguir:

- Tempo total para a inserção de todas as palavras.
- Busca de 100 elementos que não existem.
- Deleção de 100 elementos que não existem.
- Quantidade total de rotações da AVL e RBT.
- Quantidade total de rehashes das tabelas hash com encadeamente interior e exterior.
- Quantidade total de comparações da AVL, RBT e das duas tabelas hash.

Em dois casos é usado o número 100 pelo fato de que a busca ou deleção de somente um aconteceria de maneira instantânea, fazendo com que o tempo totalizasse 0s. Em seguida, foram feitas comparações entre cada tipo de dicionário, discutindo sua eficiência no contexto do problema.

Para contabilizar a quantidade de tempo total das operações, foi usada a biblioteca `<ctime>`, onde era possível salvar o valor do clock atual em uma variável `clock incial` antes de iniciar a operação. Quando a operação era finalizada, o cálculo de `clock_atual - clock_incial` dava o intervalo de clocks para a operação que foi processada. Em seguida, a quantidade de clocks era convertida para segundos.

7.3. Tempo total de inserção de todas as palavras

Para esse tópico foi utilizado somente um texto, o ”kjv bible”, que é o maior dentre todos os disponíveis.

7.3.1. AVL vs RBT

Na tabela a seguir é mostrada cada métrica das implementações.

Estrututra	Total inserção	Rotações	Comparações	Buscar	Deletar
AVL	371.33 ms	10315	7418111	0.77 ms	0.73 ms
RBT	468.52 ms	8560	8295534	0.92 ms	0.86 ms

Table 1. Tabela de rotações e comparações

A princípio, eu já esperava que a quantidade de rotações na AVL fosse superior à da RBT, pelo fato de que a AVL tem um balanceamento mais restrito. Em contraposição, a RBT teve mais comparações porque a altura da árvore criada para RBT deveria ser maior do que na AVL, sendo necessário ”descer” mais nós para encontrar um valor. No começo, eu desconfiei que a implementação da minha RBT estaria errada pelo motivo de que ela demorou mais tempo na inserção do que na AVL. Normalmente, a RBT demoraria menos tempo para inserir valores pelo fato de que ela usaria menos rotações do que a AVL. Entretanto, como utilizei somente o operador de `[]`(colchetes) para inserir, no pior caso, ele faria duas buscas, e na tabela acima a AVL tem tempo menor para buscar um elemento. Como na AVL a árvore está bem mais balanceada, sua busca acaba sendo melhor que na RBT, justificando seu menor tempo na inserção na tabela.

7.3.2. Hash Encadeamento Fechado vs Hash Encadeamento Aberto

Como foi citado anteriormente, havia um problema na OpenHash que fazia com que o tempo total fosse de aproximadamente 55s. Vou usar esse tempo para comparar com a nova Hash Aberta e a Hash Fechada. Lembrando que a antiga Hash Aberta usava somente duas bandeiras: `EMPTY` e `BUSY`. Segue uma tabela com as métricas de cada estrutura de dado.

Estrututra	Total inserção	Rehashes	Comparações	Buscar	Deletar
Hash Fechado	322.62 ms	8	955393	0.01 ms	0.01 ms
Hash Aberto	328.07 ms	8	1162665	0.04 ms	0.04 ms

Table 2. Tabela de rotações e comparações

É possível observar que o tempo $55\text{s} = 55000\text{ms}$ é bastante elevado para o tempo total de inserção nas tabelas. Por esse motivo, a mudança no Hash Aberto foi crucial para a eficiência do programa. Os tempos totais de inserção em ambos os tipos de hashes foram quase iguais. Isso pode ser explicado pelo fato de que a quantidade de vezes que o Hash Aberto fez sondagens(”pulou”) na sua tabela seja quase igual à quantidade de vezes que o Hash Fechado precisou iterar na sua lista encadeada; justificando a diferença de quase 200 mil comparações.

Como o Hash Aberto teria que fazer sondagens até o final da tabela, o Hash Fechado teve um tempo de busca e deleção menor. Para o Hash Fechado, no pior caso, ele teria que percorrer um slot inteiro, sendo esse slot o maior da tabela. Porém, como o fator de carga do Hash Fechado era menor que 0.75, era garantido que os elementos dos slots não ficariam sobrecarregados. Pode-se observar que a quantidade de rehashes foi igual. Isso pode ser explicado pelo fato de que o fator de carga de ambas as estruturas é próximo, enquanto o do Hash Aberto é 0.80, o do Hash Fechado é 0.75.

7.4. Conclusão das 4 ED’s

O Hash Fechado conseguiu alcançar o menor tempo para inserção total, busca e deleção. Porém, ele teve que fazer mais comparações do que a AVL e a RBT. Desse modo, podemos concluir que, de todas as estruturas de dados, o Hash Fechado é o melhor para a inserção, deleção e busca. O Hash Fechado também foi mais simples de implementar dentre todas as estruturas, não sendo necessário manusear ponteiros durante sua implementação. Uma desvantagem do Hash Aberto é que não é possível obter seus elementos ordenados, sendo necessária uma ordenação de maneira exterior(guardar em um vetor depois ordenar) nos seus elementos. As RBT e AVL podem ser úteis para isso, sendo que sua busca para o menor e maior elemento ocorre de maneira mais rápida, bem como a obtenção dos seus elementos ordenados.

8. Arquivos

O projeto está separado entre as pastas:

- **bin**: Onde está armazenado os executáveis das estruturas.
- **books**: Livros em formato `txt` para serem lidos.
- **count_file**: As frequência das palavras do texto junto com as métricas.

- **scripts**: O executável para contar as frequências.
- **src**: As implementações das estruturas.

Alguns dos arquivos citados acima foram criados para facilitar o uso das estruturas nos textos disponíveis.

8.1. Pasta `bin`

Nessa pasta estão presentes os executáveis das quatro estruturas de dados. Cada uma foi criada com o comando:

```
g++ -Wall -Wextra main.cpp -o freq_<nome da estrututa>
-licuuc -licui18n -licuio
```

sendo as bandeiras `-licuuc`, `-licui18n` e `-licuio` necessárias para que a biblioteca ICU funcione corretamente. Esse comando deve ser rodado nas pastas **src/<estutruta>**, onde estão presentes os arquivos `main.cpp`. Depois dessa última etapa, deve-se mover o arquivo executável criado para a pasta `bin`. Isso será necessário se houver alguma alteração nas estruturas.

8.2. Pasta scripts

Nessa pasta está o arquivo `freq`, responsável por ler qualquer livro em alguma estrutura, desde que este último esteja presente na pasta `books`.

O `freq` tem um código bash que lê a linha de comando na forma:

```
./freq <alv|rbt|hash|openhash> <book>.txt
```

Esse arquivo coleta o diretório onde está sendo executado e, de acordo com a estrutura mencionada, ele processa alguns dos quatro executáveis da pasta `bin`. É passado o livro nos argumentos do executável e depois é salvo um `txt` na pasta `count_file` de acordo com a estrutura que foi selecionada. Note que, como o `freq` coleta o diretório, ele consegue navegar entre as pastas do projeto.

Para rodar o `freq` é necessário estar na pasta `scrips`. Caso dê acesso inválido, aplique o comando:

```
chmod +x freq
```

Para autorizar a execução na sua máquina.

9. Conclusão

Foi interessante implementar a estrutura de Árvore Vermelho-Preto já que somente tinha visto em teoria e, no máximo, em pseudocódigo. Minhas principais dificuldades aconteceram na parte 2 do projeto, onde tive que aprender a manusear caracteres com bibliotecas nunca vistas. Também teve a seção de fazer os comandos para que cada estrutura executasse com algum livro escolhido. Um problema que foi interessante de se resolver se dava porque era necessário navegar entre os diretórios do projeto, já que para cada máquina o destino do arquivo podia ser diferente. Uma solução foi salvar o diretório de onde o executável do `bash` estava e, a partir disso, ir para as outras pastas. Minhas considerações finais foram que colocar em prática as estruturas dadas na disciplina ajudou a compreender melhor as suas aplicações e seu funcionamento.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2013). *Algoritmos: Teoria e Prática*. Elsevier, Rio de Janeiro, 3 edition. Tradução da 3ª edição norte-americana.

Programiz (n.d.). C++ clock() - c++ standard library. `https://www.programiz.com/cpp-programming/library-function/ctime/clock`. Acessado em 14 de julho de 2025.

[Cormen et al. 2013] [Programiz nd]